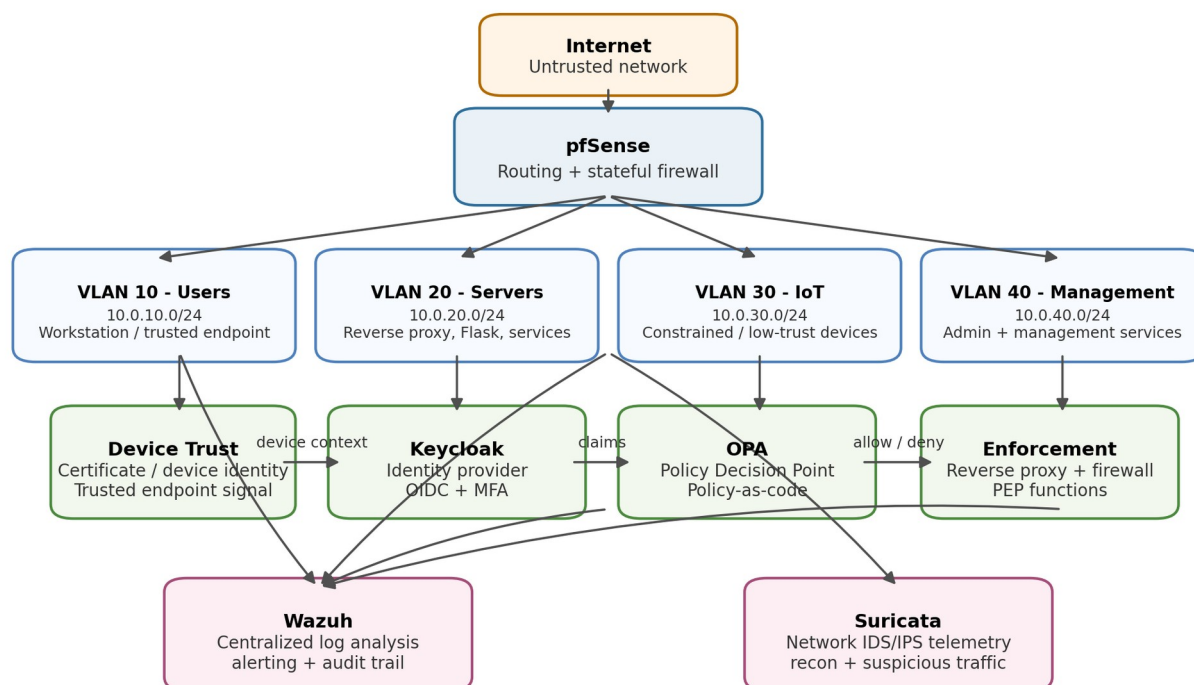


ZERO TRUST ARCHITECTURE HOME LAB

Design, Implementation, Security Validation, and Standards Mapping

Technical Project Report



Logical view: access is not trusted because of network location; identity, device context, policy, enforcement, and telemetry work together.

Figure 1. High-level logical architecture of the Zero Trust home lab.

Project overview

A resource-centric Zero Trust lab using segmented network zones, centralized identity with MFA, device trust, policy-as-code, enforcement points, and centralized security telemetry. The design is aligned conceptually to NIST SP 800-207 and mapped to CISA Zero Trust Maturity Model v2.0.

Author: Rushi Patel

Report date: September 2026

Project Scope

I designed and implemented this Zero Trust home lab to study how identity, device context, network segmentation, explicit authorization policy, and continuous monitoring can work together to reduce implicit trust. My implementation uses VirtualBox for virtualization, pfSense for network enforcement, Keycloak for identity and MFA, Open Policy Agent (OPA) for authorization decisions, a reverse proxy/application enforcement layer, Wazuh for centralized security monitoring, Suricata for network detection, and Kali Linux for controlled validation testing. This report documents the architecture, implementation decisions, control logic, threat model, validation approach, standards alignment, and lessons I took from the project.

Field	Value
Project type	Cybersecurity home lab / portfolio project
Primary domain	Zero Trust Architecture, network security, IAM, policy-as-code, detection and monitoring
Reference architecture	NIST SP 800-207 Zero Trust Architecture
Implementation guidance	NIST SP 1800-35 Implementing a Zero Trust Architecture
Maturity mapping	CISA Zero Trust Maturity Model v2.0
Validation scope	Controlled testing only against personally owned lab systems

Contents

1. Executive Summary
 2. Problem Statement and Security Objectives
 3. Zero Trust Design Principles and Standards Baseline
 4. Lab Scope, Assumptions, and Trust Boundaries
 5. Architecture Overview
 6. Network Segmentation and pfSense Enforcement
 7. Identity, Authentication, and MFA
 8. Device Trust and Context-Aware Access
 9. Policy Decision and Enforcement Architecture
 10. Protected Applications and Data Flows
 11. Centralized Logging, Detection, and Visibility
 12. Threat Model
 13. Security Policy and Access-Control Matrix
 14. Validation and Adversarial Test Plan
 15. Security Analysis by Attack Scenario
 16. NIST SP 800-207 Alignment
 17. CISA Zero Trust Maturity Model Mapping
 18. Operational Security and Administration
 19. Limitations and Future Enhancements
 20. GitHub Repository and Publication
 21. Resume and Interview Summary
- Appendix A. Representative Configuration and Policy Excerpts
- Glossary
- References

1. Executive Summary

Traditional home and small-lab networks often rely on a broad trust assumption: once a host is connected to the internal LAN, it can reach many other internal systems with little additional verification. I built this project to replace that assumption with a layered model in which access is constrained by the requested resource, user identity, device context, explicit policy, and continuous observation. My main goal was to make network location only one signal in the access decision rather than the reason a request is trusted.

I built the implementation around five major control areas. I used pfSense for routing, segmentation, and default-deny enforcement between user, server, IoT, and management zones. I used Keycloak for centralized authentication, role information, OpenID Connect integration, and multi-factor authentication. I added device identity as an additional signal for sensitive access, used OPA to evaluate authorization policy as code, and separated those policy decisions from the components that enforce them. Finally, I used Wazuh and Suricata to centralize visibility across authentication, endpoint, application, firewall, and network events.

I did not treat VLANs alone as Zero Trust. I used segmentation as one enforcement layer inside a larger decision system. Being connected to an internal subnet does not automatically give a user access to a server, database, or management interface. For sensitive paths, I designed access to require successful authentication, the correct role, an acceptable device state, and an explicit policy decision for the requested resource. I then centralized the resulting security telemetry so I could review access decisions and investigate suspicious behavior.

NIST SP 800-207 defines Zero Trust as a shift away from implicit trust based on network location and toward protection of users, assets, and resources, with authentication and authorization of the subject and device occurring before a session is established [1]. NIST SP 1800-35, finalized in June 2025, provides implementation-oriented guidance and documents multiple example ZTA implementations [2]. I used those principles as the architectural baseline for my lab while keeping the implementation small enough to build, test, and explain on a home-lab platform.

Security goal	How the lab addresses it
Eliminate implicit LAN trust	Inter-zone traffic is restricted; access depends on explicit policy rather than membership in an internal subnet.
Reduce lateral movement	User, server, IoT, and management systems are isolated with default-deny firewall policy.
Strengthen authentication	Centralized identity and MFA reduce reliance on password-only access.
Add device awareness	Approved-device identity is evaluated alongside user identity for sensitive access.
Apply least privilege	Roles and resource-specific policies limit users and systems to required actions.
Centralize decisions	OPA externalizes authorization logic as policy-as-code.
Create visibility	Wazuh and Suricata centralize host, authentication, firewall, and network telemetry.
Validate controls	Kali Linux is used to perform authorized reconnaissance, credential, untrusted-device, and lateral-movement tests.

2. Problem Statement and Security Objectives

2.1 Problem Statement

A conventional flat network makes IP reachability a major determinant of trust. If an attacker compromises a workstation, steals a password, or gains control of an IoT device, that foothold may provide opportunities to enumerate adjacent systems, reach management interfaces, and move laterally. I designed my home lab around the opposite assumption: a request should not be trusted simply because it originates from an internal network. I required the request to satisfy the relevant identity, device, network, and resource policy before access was permitted.

2.2 Project Objectives

1. Design a segmented network with clearly separated user, server, IoT, management, and optional security-monitoring zones.

2. Use pfSense to enforce explicit least-privilege traffic rules between zones and to restrict access to management services.
3. Centralize human identity through Keycloak and protect interactive access with MFA.
4. Use OIDC-based application authentication so the application does not maintain an isolated local identity model.
5. Introduce device trust so valid credentials alone are insufficient for selected privileged actions.
6. Externalize authorization decisions through Open Policy Agent and express access requirements as policy-as-code.
7. Centralize security telemetry using Wazuh and add network detection with Suricata.
8. Validate the architecture with controlled tests for reconnaissance, stolen credentials, untrusted endpoints, lateral movement, and compromised low-trust devices.
9. Map the resulting controls to NIST SP 800-207 concepts and CISA Zero Trust maturity pillars.
10. Produce a reproducible, evidence-oriented portfolio artifact suitable for GitHub, resume use, and technical interviews.

2.3 Security Success Criteria

ID	Criterion	Verification approach
SC-01	User network cannot freely initiate arbitrary connections into server, IoT, or management networks.	pfSense rule review + connection tests
SC-02	Web application requires centralized authentication.	Unauthenticated request redirects to IdP or is denied
SC-03	Protected login requires a second factor.	Password-only login test
SC-04	Sensitive management access is denied from an untrusted device.	Valid account from non-enrolled Kali endpoint
SC-05	Ordinary user role cannot access privileged resources.	Role-based request tests
SC-06	Application/database traffic is limited to explicitly required paths.	Firewall/connection verification
SC-07	IoT network cannot initiate access to protected internal zones.	Ping/TCP/scan tests from IoT test host
SC-08	Authentication, firewall, host, and network security events are centrally observable.	Wazuh/Suricata log review
SC-09	Policy decisions can be reviewed as code rather than only embedded in application logic.	OPA policy repository review

3. Zero Trust Design Principles and Standards Baseline

3.1 NIST SP 800-207 Baseline

I used NIST SP 800-207 as the primary conceptual baseline for the project. Its resource-centric approach helped me separate the decision about whether access should be allowed from the mechanisms that actually enforce the decision. I also used its emphasis on authenticating and authorizing both the subject and device before resource access as a guide for the identity and device-trust portions of the lab [1].

NIST concept	Interpretation in this lab
Resources are the focus	The web application, database, SSH management interface, identity service, and monitoring services are treated as individual protected resources.
No implicit trust by location	A device on the internal user VLAN is not automatically authorized to reach server or management resources.
Per-session/per-resource access	Access decisions are scoped to requested resources and policy conditions rather than creating broad internal trust.
Dynamic policy	Identity claims, role, device trust, source zone, and resource can be inputs to policy decisions.
PDP/PEP separation	OPA represents the policy decision function; reverse proxy/application/firewall components enforce decisions.
Continuous diagnostics and monitoring	Wazuh and Suricata collect security events so decisions and behavior remain observable.

3.2 NIST SP 1800-35 Implementation Guidance

NIST SP 1800-35 was finalized in June 2025 as an implementation-oriented practice guide. It documents 19 example Zero Trust implementations built with industry collaborators and demonstrates common use cases across on-premises and cloud resources [2]. I used this guide mainly as confirmation that Zero Trust should be treated as an architecture assembled from identity, policy, enforcement, network, application, device, and monitoring capabilities rather than as a single product.

3.3 CISA Zero Trust Maturity Model v2.0

CISA ZTMM v2.0 organizes Zero Trust maturity around five pillars - Identity, Devices, Networks, Applications and Workloads, and Data - with visibility and analytics, automation and orchestration, and governance applied across those pillars [3]. In Section 17, I map my controls to those areas to show which Zero Trust capabilities the project covers and where my home-lab implementation remains intentionally simplified. I use the model as an architectural reference, not as a claim of federal compliance or enterprise maturity.

Important distinction

This project is "aligned with" or "mapped to" Zero Trust guidance. It should not be described as NIST-certified, CISA-certified, or compliant unless an appropriate independent compliance process exists. NIST SP 800-207 is an architectural publication, not a certification program.

4. Lab Scope, Assumptions, and Trust Boundaries

4.1 Reference Technology Stack

Layer	Technology	Role in the project
Virtualization	Oracle VirtualBox	Hosts the lab VMs and provides isolated virtual network adapters.
Network enforcement	pfSense	Layer 3 routing, stateful firewalling, inter-zone controls, management-plane restrictions, and security logs.
Identity	Keycloak	Central identity provider, OIDC integration, roles, MFA/TOTP, authentication events.
Policy	Open Policy Agent	Policy decision point / policy-as-code engine for contextual authorization.
Application enforcement	Reverse proxy + application middleware	Policy enforcement point for protected HTTP requests.
Application	Flask test application	Protected workload used to exercise identity and authorization decisions.
Data	MySQL-compatible database	Protected data tier reachable only from explicitly authorized application path.
SIEM / host visibility	Wazuh	Central log collection, decoding, rule evaluation, dashboards, and alerting.
Network detection	Suricata	IDS/IPS-capable network analysis for reconnaissance and suspicious flows.
Validation	Kali Linux	Authorized testing host for scanning, connection, and identity-control validation.

4.2 Network Trust Zones

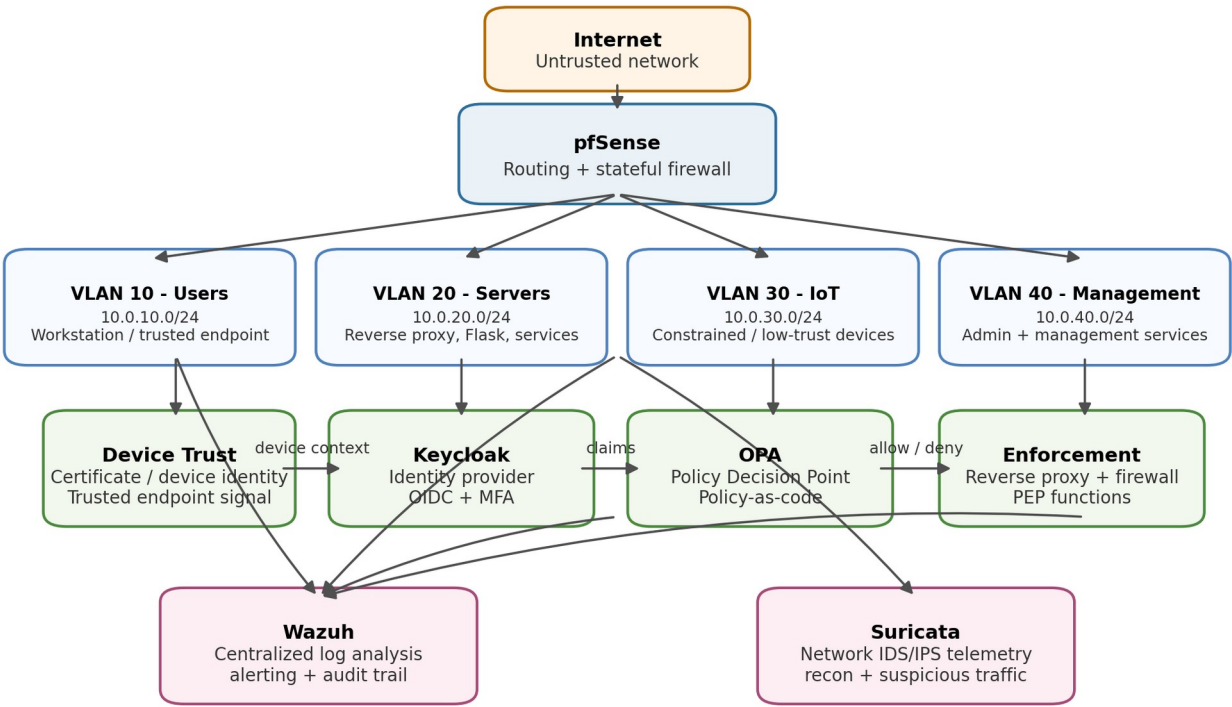
Zone	Example subnet	Trust posture	Representative assets
User	10.0.10.0/24	Authenticated but not broadly trusted	User workstation / test client
Server	10.0.20.0/24	Protected workload zone	Reverse proxy, Flask app, application services
IoT	10.0.30.0/24	Low trust / constrained	IoT test VM or simulated device
Management	10.0.40.0/24	High sensitivity	Administrative endpoints and management interfaces
Security (optional)	10.0.50.0/24	Restricted security operations	Wazuh server/dashboard, sensors

4.3 Trust Boundaries

I identified seven main trust boundaries in the design: (1) Internet to firewall, (2) every inter-VLAN transition, (3) the identity boundary at Keycloak, (4) the authorization boundary at OPA, (5) the application enforcement boundary at the reverse proxy or middleware, (6) application to database, and (7) the monitoring and administration boundary around Wazuh and management interfaces. At each boundary, I required an explicit control instead of assuming that the source was safe because of its location.

4.4 Assumptions

- I ran the lab on a personally controlled host and did not expose it as a production public service.
- I limited all scanning and adversarial testing to IP ranges and resources inside my authorized lab environment.
- The IP ranges and hostnames shown in the report document the logical design; the same architecture can be reproduced with different private addressing.
- I represented device trust with an approved device identity or certificate. I treat that as a simplified trust signal that can later be extended with richer endpoint posture data.



Logical view: access is not trusted because of network location; identity, device context, policy, enforcement, and telemetry work together.

Figure 2. Reference logical architecture and primary trust boundaries.

5. Architecture Overview

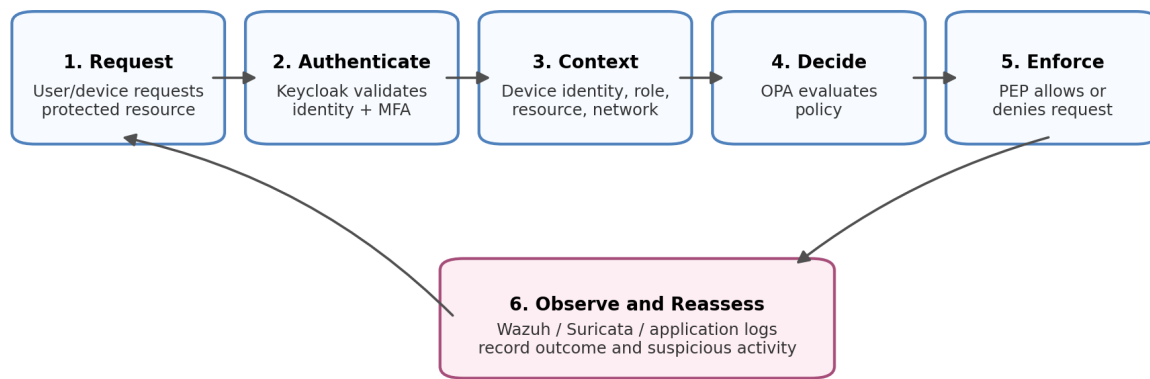
5.1 Architectural Layers

I intentionally layered the architecture so that failure of one control would not automatically create unrestricted access. Network segmentation limits reachability, Keycloak verifies user identity, MFA reduces the value of a stolen password, device identity adds endpoint context, OPA evaluates authorization, and the enforcement points translate policy decisions into access outcomes. I then used Wazuh and Suricata to record and analyze activity across those layers. Each control addresses a different failure mode, which is the main defense-in-depth lesson I wanted the lab to demonstrate.

Plane	Primary function	Failure it is intended to contain
Data plane	Carries permitted application and network	Prevents direct exposure of sensitive

	traffic.	resources where no path is required.
Identity plane	Authenticates users and issues identity/role claims.	Reduces uncontrolled local accounts and password-only access.
Policy plane	Computes authorization decisions from context.	Prevents role and access logic from being scattered or hard-coded.
Enforcement plane	Applies network and application decisions.	Stops unauthorized sessions from reaching protected resources.
Visibility plane	Collects, correlates, and surfaces events.	Reduces blind spots after attempted or successful access.
Management plane	Administers firewall, IdP, SIEM, policy, and hosts.	Separates high-risk administrative functions from ordinary user access.

5.2 Request Lifecycle



The loop is deliberate: authentication is not a permanent grant. Each resource access remains policy-controlled and observable.

Figure 3. Zero Trust access decision and observation loop.

A protected request is evaluated in stages. The client requests a resource. Keycloak establishes user identity and MFA status. Device identity and role/resource context are assembled. OPA evaluates policy and returns an authorization decision. A policy enforcement point allows or denies the request. The decision and related security activity are logged. This model keeps authentication, authorization, enforcement, and monitoring conceptually distinct, which makes the design easier to reason about and test.

5.3 Why Segmentation Alone Is Insufficient

VLANs and firewall rules are valuable, but they do not know who a human user is, whether MFA occurred, or whether a device is enrolled. Conversely, identity alone does not stop a compromised service from probing management interfaces if the network is flat. The project therefore combines network segmentation with identity-aware and policy-aware application access. This is the central architectural argument of the lab.

6. Network Segmentation and pfSense Enforcement

6.1 Role of pfSense

I used pfSense as the primary network policy-enforcement component. Netgate documents pfSense as filtering traffic and deciding which traffic is passed or blocked between networks [4]. I used that capability to make each VLAN a distinct security zone rather than using VLANs only for organization. This also follows Netgate guidance that networks with different trust levels should be segregated and that VLAN security depends on correct configuration [5].

6.2 Default-Deny Methodology

The implementation follows a deny-by-default design between internal zones. Broad rules such as "LAN to any" are removed or avoided for protected segments. Required flows are added narrowly by protocol, source, destination, and port. Netgate documentation explicitly describes removing the default LAN-to-any rule as part of a stricter LAN/DMZ ruleset [4].

Source	Destination	Service	Action	Reason
User VLAN	Reverse proxy / web app	TCP 443	Allow	Required user-facing application path
User VLAN	Database	TCP 3306	Deny	Users must not directly reach data tier
User VLAN	Management VLAN	Any by default	Deny	Administrative plane is isolated
Server VLAN	Database	TCP 3306	Allow only from app host	Application dependency
Server VLAN	Management VLAN	Any by default	Deny	Prevents server compromise from becoming admin-plane access
IoT VLAN	Internal protected zones	Any by default	Deny	Low-trust devices do not initiate internal sessions
IoT VLAN	Internet	Required outbound only	Allow selectively	Supports necessary updates/services
Management VLAN	Admin interfaces	HTTPS/SSH as required	Allow from approved admin source	Controlled administration
Any internal zone	Security monitoring	Syslog/agent-specific	Allow as required	Centralized telemetry

6.3 Management-Plane Protection

I treated the firewall GUI and other administrative interfaces as high-value resources. I restricted them to the management zone or designated administrative hosts. Netgate recommends limiting management access with firewall rules and restricting the source hosts that can reach the GUI [4]. I applied the same principle to Keycloak administration, Wazuh administration, OPA policy management, and server SSH.

6.4 Stateful Filtering and Rule Placement

pfSense firewall rules are stateful by default and interface rules are applied to traffic entering an interface [4]. This matters when documenting the ruleset: a user-to-server connection should be controlled on the user-side interface where the traffic enters the firewall. Return traffic is handled by the established state. Understanding this flow avoids creating redundant or contradictory rules and demonstrates that the segmentation was intentionally designed rather than configured through trial and error.

6.5 Network Verification

I validated network controls from both allowed and denied paths. Reaching the protected application was only the positive case; I also tested that prohibited destinations remained unreachable. The following commands are representative of the checks I used inside the isolated lab:

```
# Validate local addressing and routes
ip addr
ip route

# Test a permitted HTTPS path
curl -kI https://<protected-app>

# Test that a prohibited port is not reachable
nc -vz <database-ip> 3306

# Enumerate only the authorized lab subnet
nmap -sT -Pn <authorized-lab-subnet>
```

Supporting project evidence

Export or screenshot the pfSense interface assignments, VLAN/subnet configuration, firewall rules, and firewall logs showing at least one permitted and one denied inter-zone test. Redact public IPs or secrets before publishing.

7. Identity, Authentication, and MFA

7.1 Centralized Identity

Keycloak serves as the central identity provider for the lab. The application delegates user authentication to Keycloak through OpenID Connect rather than storing a separate password database. Keycloak documents OIDC support for browser-based authorization code flows and exposes standard authorization, token, logout, user-info, and certificate endpoints [6]. This allows the application to consume signed identity claims instead of implementing authentication from scratch.

7.2 Authentication Flow

1. The user requests the protected application.
2. The application or reverse proxy detects that no valid authenticated session is present.
3. The browser is redirected to the Keycloak authorization endpoint.
4. Keycloak validates the primary credential and applies the configured MFA flow.
5. After successful authentication, Keycloak returns an authorization code through the registered callback path.
6. The application exchanges the code for tokens and validates issuer, audience, signature, expiration, and relevant claims.
7. Authorization context such as role and MFA state is supplied to the policy/enforcement layer before the resource is exposed.

7.3 Multi-Factor Authentication

The reference configuration uses time-based one-time password (TOTP) as the second factor. Keycloak provides OTP policies and authentication flows that can make a second factor required [6]. For the lab, the important control is not the specific authenticator brand but the access property: a correct password alone should not be sufficient for a protected interactive session.

Test	Primary credential	Second factor	Expected decision
Valid user + MFA	Valid	Valid	Authentication succeeds; authorization still evaluated
Valid password only	Valid	Missing/invalid	Authentication denied or incomplete
Invalid password	Invalid	N/A	Authentication denied
Disabled/unauthorized account	Potentially valid	Potentially valid	Access denied based on account/policy state

7.4 Role Model

I kept the role model small so I could reason about every authorization path while still demonstrating least privilege. I deliberately kept roles separate from network location so that an IP address or VLAN did not become a substitute for identity.

Role	Purpose	Typical privileges
Administrator	Manage protected infrastructure.	Management interface and SSH only from approved admin context; application administration.
User	Consume the normal protected application.	HTTPS access to approved user-facing application routes; no database or management access.
Guest/Test	Negative testing and minimal-access scenarios.	No sensitive resources; used to verify denial behavior.

7.5 Token and Session Security Considerations

- Use HTTPS for identity and application traffic; avoid exposing bearer tokens over plaintext transport.
- Validate token issuer, audience, signature, expiration, and required claims at the enforcement point.
- Use short-lived access tokens and appropriate session timeouts for a lab that demonstrates re-evaluation rather than indefinite trust.
- Protect client secrets and signing material; do not commit secrets, private keys, or exported credentials to GitHub.
- Keep the Keycloak administrative console isolated from ordinary user networks.

8. Device Trust and Context-Aware Access

8.1 Why Device Identity Matters

If authorization evaluates only the username, stolen credentials can still be used from an attacker-controlled endpoint. The device-trust layer introduces a second identity dimension: who is the user, and what endpoint is making the request? For sensitive paths, the policy can require both a valid user identity and an approved device identity.

8.2 Lab Device-Trust Model

At home-lab scale, I represented device trust with an approved endpoint identity that can be backed by a certificate or other verifiable device signal. This let me test an important case: a Kali VM can possess valid test credentials and still fail a sensitive management policy because the request does not include an approved device identity.

Context attribute	Example value	Use in decision
user.role	admin	Determines whether privileged function is even eligible.
user.mfa	true	Requires strong authentication before sensitive access.
device.trusted	true/false	Prevents valid credentials from automatically authorizing unknown endpoints.
source.zone	user / management / IoT	Adds network context without treating network location as sufficient trust.
resource	web / db / ssh / admin-console	Scopes privileges to the requested resource.
action	read / administer / connect	Allows more granular decisions than simple login/no-login.

8.3 Device Trust Is Not Full Endpoint Detection

A certificate proves possession of a device identity; it does not by itself prove that the device is healthy. A more mature design would add posture signals such as OS version, disk encryption, endpoint protection state, patch level, or EDR health. This limitation is documented explicitly so the project does not overstate what certificate-based device trust can guarantee.

9. Policy Decision and Enforcement Architecture

9.1 OPA as the Policy Decision Point

Open Policy Agent is a general-purpose policy engine designed to decouple policy decision-making from enforcement. OPA accepts structured input, evaluates policy and data, and returns a decision [7]. Its deployment guidance explicitly describes OPA as a policy decision point serving applications that act as policy enforcement points [8]. This maps cleanly to the conceptual separation described in Zero Trust architectures.

9.2 Policy Inputs

A typical policy request can combine identity, device, network, resource, and action context. Keeping this data structured makes the policy reviewable and testable.

```
{
```

```

"user": {
  "name": "test-admin",
  "role": "admin",
  "mfa": true
},
"device": {
  "trusted": true,
  "device_id": "lab-laptop-01"
},
"network": {
  "source_zone": "management"
},
"resource": "ssh-management",
"action": "connect"
}

```

9.3 Representative Policy-as-Code

The following excerpt represents the policy model I used for the project. I kept the example compact, but it still shows the main authorization attributes I wanted to evaluate: resource, action, role, MFA state, device trust, and network context.

```

package homelab.authz

default allow := false

# Standard user can access the protected web application only
# after MFA and from a trusted endpoint.
allow if {
  input.resource == "web-app"
  input.action == "read"
  input.user.role in {"user", "admin"}
  input.user.mfa == true
  input.device.trusted == true
}

# Administrative SSH additionally requires the management zone.
allow if {
  input.resource == "ssh-management"
  input.action == "connect"
  input.user.role == "admin"
  input.user.mfa == true
  input.device.trusted == true
  input.network.source_zone == "management"
}

```

9.4 Policy Enforcement Points

The architecture uses more than one enforcement point. pfSense enforces network reachability between zones. The reverse proxy or application middleware enforces the identity-aware decision before forwarding a protected HTTP request. SSH and administrative interfaces are also protected by network source restrictions and strong authentication. This layered design means a single permissive rule does not automatically create full application authorization.

9.5 Deny-by-Default Behavior

I wrote the OPA policy with a default-deny decision. A request is allowed only when a specific rule matches. This mirrors the firewall design and makes the authorization model easier for me to audit because every permitted path can be traced to an explicit policy condition.

10. Protected Applications and Data Flows

10.1 Three-Tier Protection Model

The protected workload is modeled as a user-facing reverse proxy/application tier and a separate database tier. The user never needs direct database connectivity. The database therefore accepts only the application dependency required for normal operation. This creates a clear demonstration of resource-level protection and limits the value of a compromised user endpoint.



10.2 Representative Allowed Data Flows

Flow	Source	Destination	Control
Application login	User endpoint	Keycloak	HTTPS + OIDC + MFA
Application request	User endpoint	Reverse proxy / app	HTTPS + session/token validation + OPA decision
Authorization query	PEP	OPA	Structured policy input; allow/deny response
Application data access	Flask app	Database	Firewall-restricted application dependency
Security logs	Hosts/services/firewall	Wazuh	Agent/syslog/log forwarding
Network telemetry	Observed network path	Suricata	IDS/IPS sensor or integration

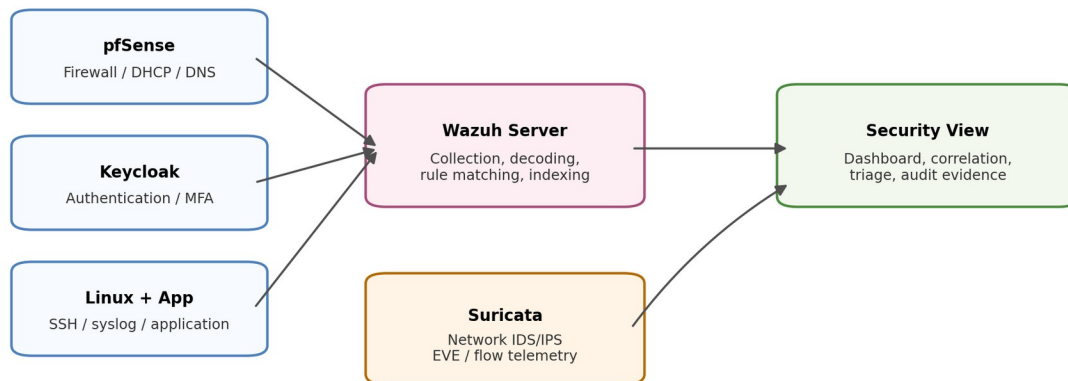
10.3 Explicitly Disallowed Data Flows

- User workstation directly to database service.
- IoT zone directly to application administration or management interfaces.
- Ordinary user role to SSH management service.
- Untrusted endpoint to sensitive administrative service even when test credentials are valid.
- Compromised application server to unrelated management resources unless a specific operational dependency exists.

11. Centralized Logging, Detection, and Visibility

11.1 Monitoring Strategy

I treated logging and detection as part of the architecture rather than as an afterthought. A Zero Trust decision is difficult to verify if I cannot reconstruct who requested a resource, from which device, what the policy decided, and what happened afterward. I used Wazuh to centralize endpoint, application, identity, and network-device events and Suricata to add network-oriented detection that complements host and identity telemetry [9][10].



Centralized visibility connects prevention to detection: policy denies, authentication events, host events, and network indicators become reviewable evidence.

Figure 4. Centralized security telemetry and analysis pipeline.

11.2 Log Sources Collected

Source	Security value	Example events
pfSense	Shows network enforcement decisions.	Blocked inter-zone connection, allowed HTTPS, management attempt
Keycloak	Shows identity lifecycle and authentication outcomes.	Login success/failure, MFA challenge, admin event
Linux server	Shows host access and system activity.	SSH authentication, sudo, service events, syslog
Flask / reverse proxy	Shows resource requests and enforcement behavior.	HTTP status, user/role, denied path, policy result
OPA decision log	Shows why authorization was granted or denied.	Input context, policy decision, policy version
Suricata	Shows network indicators and reconnaissance.	Scan pattern, suspicious flow, protocol alert

11.3 Correlation Questions

I designed the monitoring flow so I could answer questions across layers: Which user authenticated? Was MFA completed? Which device originated the request? Which resource was requested? What did OPA decide? Did pfSense allow the network path? Did Suricata observe scan behavior? Did the host record a corresponding process or SSH event? I found this cross-layer view more useful than treating each log source as an isolated dashboard.

11.4 Example Detection Use Cases

Use case	Telemetry	Analyst interpretation
Repeated failed login	Keycloak + Wazuh	Potential password guessing or account misuse; review source and account.
Network scan from user VLAN	Suricata + pfSense	Endpoint may be performing reconnaissance; compare with authorized test windows.

Denied management access	pfSense/PEP + Wazuh	Expected control if source is ordinary user or untrusted device; investigate if repeated.
Valid login followed by denied policy	Keycloak + OPA	Identity was valid, but role/device/resource policy correctly constrained access.
Unexpected server-to-management flow	pfSense + Suricata	Potential lateral movement or misconfiguration; validate dependency and block if unnecessary.

12. Threat Model

12.1 Protected Assets

- Identity provider and authentication secrets.
- Administrative interfaces and management credentials.
- Protected web application and application session/token material.
- Database contents and database credentials.
- Firewall configuration and security policy.
- OPA policies and policy-decision service.
- Security logs, Wazuh dashboard, and detection configuration.

12.2 Adversary Assumptions

For my threat model, I assumed an attacker could gain control of one low-privilege endpoint, learn one valid test password, compromise an IoT-class system, or gain execution on an application server. I evaluated whether any of those footholds automatically provided broader access. I did not model nation-state capabilities, hypervisor compromise, stolen private signing keys, or a fully compromised administrator device.

12.3 Threat Scenarios

ID	Scenario	Primary risk	Controls expected to limit it
T-01	Reconnaissance from user endpoint	Service discovery and attack-surface mapping	pfSense segmentation, Suricata visibility
T-02	Stolen password	Unauthorized login	MFA, account policy, logging
T-03	Valid credentials from unknown device	Credential replay from attacker-controlled endpoint	Device trust + OPA policy
T-04	Compromised user workstation	Lateral movement to servers/database/management	Segmentation + least privilege + per-resource policy
T-05	Compromised application server	Pivot to management plane or unrelated resources	Server-zone egress restrictions, management isolation
T-06	Compromised IoT device	Movement from low-trust zone into sensitive network	IoT isolation + explicit outbound rules
T-07	Privileged misuse	Administrator overreach	Role/resource policy, management restrictions, logs
T-08	Monitoring failure	Loss of visibility during attack	Central collection, source-health checks, protected SIEM

12.4 Abuse Cases

I also included misuse of legitimate paths in the threat model. An allowed HTTPS connection can still carry a malicious application request, so I did not rely on the firewall as the final authorization control. Likewise, a valid admin token should not be accepted by an unrelated service without the correct audience and scope, and a trusted device should not be assumed to remain healthy forever. These cases reinforced why I used multiple contextual checks rather than one gate.

13. Security Policy and Access-Control Matrix

13.1 Human Access Matrix

Identity	Device	Web app	Database	SSH / management
Administrator	Trusted	Allow after MFA	No direct user DB access	Allow only under admin policy/management

				context
Administrator	Untrusted	Deny sensitive path	Deny	Deny
User	Trusted	Allow after MFA	Deny	Deny
User	Untrusted	Deny or restrict by policy	Deny	Deny
Guest/Test	Any	Deny sensitive functions	Deny	Deny

13.2 Workload Access Matrix

Source workload	Destination	Required access	Default posture
Reverse proxy	Flask application	Application port only	Deny other server paths
Flask application	Database	DB port only	Deny unrelated networks
Database	User VLAN	No initiated sessions	Deny
IoT device	Internal protected zones	None by default	Deny
Monitoring agents/devices	Wazuh	Logging/agent protocol	Allow narrowly
Administrative host	pfSense/Keycloak/Wazuh/servers	Management ports	Allow only from approved source

13.3 Policy Rationale

I used the access matrix to make two design choices explicit. First, I separated human access from workload access: a user can be authorized to use an application without receiving direct access to its database. Second, I did not treat the administrator role as a universal bypass. Sensitive management access still requires an approved endpoint and management context, so role membership alone does not become a new form of implicit trust.

14. Validation and Adversarial Test Plan

14.1 Validation Philosophy

I validated each major control with both a positive case and a negative case. The positive case confirmed that required access still worked, while the negative case checked whether the same control blocked or detected behavior outside policy. I performed these tests only inside my isolated, personally controlled lab environment.

14.2 Test Case Catalog

Test ID	Procedure	Expected security result	Evidence
VT-01	From trusted user endpoint, browse to protected application and complete MFA.	Application becomes available only after identity and policy checks.	Browser + Keycloak event + app/PEP log
VT-02	Attempt application access without completing MFA.	Authentication is denied or remains incomplete.	Keycloak authentication event
VT-03	Use ordinary user role to request admin-only function.	Authorization denied even after valid authentication.	OPA/PEP decision log
VT-04	Use valid test account from untrusted Kali endpoint for management access.	Sensitive access denied by device/context policy.	OPA/PEP + firewall logs
VT-05	Scan server subnet from user network.	Only explicitly exposed service(s) reachable; scan behavior observable.	Nmap output + Suricata/pfSense logs
VT-06	Attempt direct user-to-database TCP connection.	Connection blocked.	Client test + pfSense block log
VT-07	From server zone, attempt unrelated management connection.	Connection blocked unless explicitly required.	Client test + firewall log
VT-08	From IoT zone, attempt connection to user/server/management zones.	Internal initiation denied.	Ping/TCP test + firewall logs
VT-09	Generate repeated invalid test logins.	Events visible centrally and suitable for alerting/correlation.	Keycloak + Wazuh dashboard
VT-10	Stop or disconnect a monitored agent in a controlled test.	Monitoring health degradation becomes visible.	Wazuh agent status

14.3 Safe Test Commands

```
# Connectivity / segmentation checks
ping -c 4 <lab-host>
```



```
nc -vz <lab-host> <port>
curl -kI https://<lab-app>

# Controlled service discovery inside the authorized lab only
nmap -sT -sV -Pn <authorized-lab-host-or-subnet>

# Authentication / SSH negative test
ssh <test-user>@<management-host>

# Review local route and neighbor state when troubleshooting
ip route
ip neigh
```

14.4 Evidence Quality

For the GitHub repository, I organize evidence around specific validation claims rather than collecting screenshots without context. I tie evidence to a test ID, source, destination, timestamp, and expected security outcome. For example, VT-06_user_to_db_blocked.png directly identifies the direct-database-access test and makes the validation trail easier to follow.

15. Security Analysis by Attack Scenario

15.1 Reconnaissance from a User Endpoint

A flat LAN often allows a compromised workstation to enumerate many internal hosts. In my design, I used pfSense to restrict inter-zone reachability so a user-zone endpoint can reach only the services I intentionally expose. I also used Suricata as an independent detection layer for scan-like behavior. My goal was not to make reconnaissance impossible; it was to make discovery different from reachability and to make reconnaissance observable.

15.2 Stolen Credentials

I treated a stolen password as an insufficient authorization signal. Keycloak MFA adds a second factor before an interactive session can complete, and OPA still evaluates the requested resource after authentication. This gave me two distinct checkpoints: authentication establishes the user session, while authorization decides whether that authenticated context may access a specific resource.

15.3 Valid Credentials from an Untrusted Device

This was one of the clearest Zero Trust tests in my lab. A test user can have a valid username and password - and potentially a valid authenticated identity - while a sensitive request is still denied because device.trusted is false. This helped me demonstrate that authenticated identity is only part of the access context and is not equivalent to universal internal access.

15.4 Lateral Movement after User Workstation Compromise

For a compromised user workstation, I relied on multiple containment controls. I did not permit direct user-to-database access, I isolated management access, and I limited the normal application path to HTTPS with identity and authorization checks. A compromised endpoint could still attack the exposed application, but the compromise would not automatically create a path to every internal resource.

15.5 Lateral Movement after Server Compromise

I treated server compromise separately because an application server legitimately needs more internal access than a user device. I minimized those dependencies by allowing the application only the database path it needed while keeping management and unrelated resources blocked. This reduces the blast radius of a web-tier foothold and makes unexpected east-west traffic easier to identify.

15.6 IoT Compromise

I placed IoT devices in a low-trust zone because these devices can have limited patching, weaker authentication, or unnecessary discovery behavior. I denied initiated access from that zone to protected internal networks and allowed only the outbound communication required for normal operation. The containment goal was to prevent compromise of a low-trust device from becoming compromise of the rest of the lab.

15.7 Monitoring and Incident Reconstruction

For each scenario, I used the visibility plane to build a chronological narrative from the source host through identity events, firewall outcomes, policy decisions, network alerts, and host/application events. This event chain helped me connect preventive controls with detective controls instead of evaluating each component in isolation.

16. NIST SP 800-207 Alignment

The following mapping shows how I translated major NIST Zero Trust concepts into concrete controls in my lab. I use it as an architectural crosswalk rather than as a certification or compliance claim.

Zero Trust principle / component	Lab implementation	Supporting evidence
No implicit trust based on network location	Internal VLAN membership does not grant unrestricted inter-zone access.	pfSense rules + denied connection test
Protect individual resources	Application, database, SSH, IdP, SIEM, and management interfaces are separately controlled.	Architecture + access matrix
Authenticate subject and device	Keycloak authenticates user; device identity is evaluated for sensitive access.	IdP event + policy input
Dynamic access policy	OPA uses role, MFA, device, source zone, resource, and action as decision inputs.	Rego policy + decision log
Policy Decision Point	OPA performs authorization evaluation.	OPA service/configuration
Policy Enforcement Point	Reverse proxy/app middleware and pfSense enforce decisions.	PEP logs + firewall rules
Least privilege	Only required application/network paths are permitted.	Access matrix + firewall rules
Continuous monitoring	Wazuh and Suricata centralize identity, endpoint, firewall, and network events.	Dashboard + alert/log examples
Assume breach / limit blast radius	User, IoT, server, and management compromise scenarios are contained by segmentation and policy.	Validation test results

16.1 Conceptual PDP/PEP Mapping

NIST SP 800-207 describes logical Zero Trust components including policy decision and enforcement functions [1]. In my implementation, OPA most directly represents the policy decision function. I use the reverse proxy or application middleware as an application-layer enforcement point and pfSense as a network-layer enforcement point. Keycloak supplies identity context used by policy, while Wazuh and Suricata provide telemetry for investigation and for possible automated response in a future iteration.

16.2 Resource-Centric Interpretation

One of my most important design choices was to describe protected objects as resources rather than only as networks. "Server VLAN" is not the final authorization target; the protected web route, database service, SSH endpoint, or administrative console is. I use network segmentation to reduce the reachable attack surface and resource policy to decide whether a specific request should succeed.

17. CISA Zero Trust Maturity Model Mapping

CISA ZTMM v2.0 uses five pillars with cross-cutting visibility and analytics, automation and orchestration, and governance [3]. My home lab touches all five pillars conceptually, but I do not present it as an enterprise-maturity implementation. The mapping below shows the capabilities I implemented and the areas I intentionally left simplified.

CISA pillar	Implemented lab capability	Current maturity limitation
-------------	----------------------------	-----------------------------

Identity	Central IdP, OIDC, roles, MFA, authentication logs.	Limited user lifecycle, risk scoring, federation, and enterprise governance.
Devices	Trusted-device identity/certificate as policy input.	No full EDR/MDM health posture or continuous device compliance.
Networks / Environment	Segmented zones, default-deny inter-zone policy, restricted management paths, IDS visibility.	Single-site virtual lab; no enterprise SASE/SD-WAN or broad encrypted traffic analytics.
Applications & Workloads	Protected application, reverse proxy/PEP, policy-as-code, restricted app-to-DB path.	Small number of workloads; no large-scale service mesh or CI/CD policy gates.
Data	Database isolated from direct user access; access mediated through application.	No formal data classification/DLP/key-management lifecycle.
Visibility & Analytics	Wazuh + Suricata + identity/firewall/application logs.	Lab-scale correlation; no mature UEBA/SOAR program.
Automation & Orchestration	Policy decisions can be automated through OPA and PEP integration.	Response automation remains limited and should be expanded cautiously.
Governance	Documented access matrix, threat model, test plan, and standards mapping.	No enterprise policy committee, formal risk acceptance, or audit program.

17.1 Maturity Interpretation

I did not use the CISA mapping to assign a formal maturity level to my home network. I used it to check that my project covered Zero Trust as a multi-pillar architecture rather than treating it as a firewall feature. The strongest areas in my lab are segmentation, identity, policy enforcement, and visibility; the largest gaps are enterprise-grade device posture, formal data governance, and automated response at scale.

18. Operational Security and Administration

18.1 Administrative Separation

I restricted administrative interfaces to the management zone or specifically approved administrative endpoints. This includes pfSense, Keycloak administration, Wazuh administration, OPA policy management, hypervisor management, and SSH. I also kept management credentials separate from ordinary application accounts.

18.2 Secrets Management

- I keep private keys, service-account keys, database passwords, OAuth client secrets, recovery codes, and environment files containing secrets out of the public GitHub repository.
- I use .gitignore rules for .env files, *.pem and *.key material, secret exports, and local configuration backups that may contain credentials.
- I publish only sanitized example files such as .env.example with placeholder values.
- If a credential is ever captured in a screenshot or committed to version control, I treat it as exposed and rotate it.
- I keep identity-signing keys and certificate private keys outside the public repository.

18.3 Change Management for a Home Lab

I used lightweight change control for the project. I versioned network-rule documentation, OPA policy files, sanitized configuration snippets, and report material so I could track what changed, why I changed it, which security assumption it affected, and how I validated it. Git commits provide the audit trail for those changes.

18.4 Backup and Recovery

I kept sanitized backups of the configuration material that is safe to preserve with the project, including firewall documentation, identity configuration notes, OPA policy files, and architecture documentation. I kept secrets separate. My recovery goal is to be able to reconstruct the architecture from the repository plus private secret material without depending on undocumented GUI state.

18.5 Monitoring Health

I treated the monitoring system itself as a protected asset. A disconnected Wazuh agent, missing firewall logs, or a disabled Suricata sensor can create a false sense of security, so I included data-source health in my operational checks in addition to reviewing security alerts.

19. Limitations and Future Enhancements

19.1 Current Limitations

Limitation	Security implication	Improvement
Device trust is simplified	Certificate possession does not prove current endpoint health.	Add MDM/EDR-style posture signals or scripted health attestation.
Single-host virtualization	Hypervisor compromise could bypass several logical boundaries.	Move critical components to separate hardware or a small multi-node lab.
Lab-scale identity	Few users/roles reduce lifecycle complexity.	Add joiner/mover/leaver flows, service identities, and separate admin accounts.
Limited data governance	Database segmentation does not equal complete data security.	Add classification, encryption-key lifecycle, backup protection, and DLP concepts.
Limited automated response	Alerts may require manual investigation.	Add carefully scoped response playbooks for repeated auth failures or known malicious hosts.
No production availability objective	Single instances may be single points of failure.	Add HA identity, monitoring, and firewall patterns if resilience becomes a goal.
No quantitative performance study	Policy/enforcement overhead is not benchmarked.	Measure latency, CPU/RAM, rule-evaluation time, and log-ingest impact under repeatable load.

19.2 High-Value Future Enhancements

1. Add mutual TLS for service-to-service authentication and certificate rotation.
2. Integrate richer device posture signals and deny access when health checks fail.
3. Use short-lived SSH certificates or an identity-aware bastion instead of long-lived SSH keys.
4. Add a secrets manager so application secrets are retrieved at runtime rather than stored in configuration files.
5. Create automated OPA unit tests and run them in a CI pipeline before policy changes are accepted.
6. Add infrastructure-as-code for reproducible network and server deployment where practical.
7. Expand Wazuh dashboards with test-ID tags and create a repeatable incident timeline for each validation scenario.
8. Add TLS termination/certificate management and document trust chains.
9. Create a controlled remote-access path using an identity-aware overlay rather than exposing management ports to the Internet.
10. Perform quantitative testing of authorization latency, log-ingest delay, and policy-change propagation.

19.3 What Would Make the Project Enterprise-Like

If I expanded this into a more enterprise-like implementation, I would add authoritative asset inventory, MDM/EDR posture, phishing-resistant authentication, privileged access management, service identity, certificate lifecycle automation, data classification, enterprise log retention, risk-based policy, centralized secrets management, high availability, CI/CD policy testing, and formal governance. The value of my home lab is that it establishes the architectural relationships those services would extend.

20. GitHub Repository and Publication

20.1 Repository Structure

```
zero-trust-home-lab/
├── README.md
├── docs/
│   └── Zero_Trust_Architecture_Home_Lab_Report.pdf
├── diagrams/
│   ├── architecture.png
│   └── access-decision-flow.png
```

```

├── telemetry-flow.png
├── policy/
│   ├── homelab.rego
│   └── tests/
├── firewall/
│   ├── network-plan.md
│   └── sanitized-rule-summary.md
├── identity/
│   └── keycloak-configuration-notes.md
├── monitoring/
│   ├── wazuh-log-sources.md
│   └── suricata-notes.md
├── evidence/
│   ├── VT-01_authentication-mfa.png
│   ├── VT-04_untrusted-device-denied.png
│   ├── VT-06_user-to-db-blocked.png
│   └── VT-09_wazuh-auth-events.png
├── scripts/
│   └── validation-commands.md
├── .gitignore
└── LICENSE

```

20.2 README Opening

README summary

I built this NIST SP 800-207-aligned Zero Trust home lab using pfSense network segmentation, Keycloak identity and MFA, device-aware authorization, Open Policy Agent policy-as-code, Wazuh centralized monitoring, Suricata network detection, and Kali Linux validation testing. I designed the architecture around explicit per-resource access, least privilege, default-deny segmentation, and centralized security telemetry instead of implicit trust based on LAN location.

20.3 README Sections

- Architecture diagram and concise project overview.
- Problem statement explaining why I moved away from a flat trusted LAN.
- Technology stack with the purpose of each component.
- Network zones and access matrix.
- Identity/MFA and device-trust flow.
- OPA authorization design and a representative policy example.
- Validation table linking each test ID to supporting evidence.
- NIST SP 800-207 and CISA ZTMM mapping.
- Limitations and future improvements.
- References and a statement that all security testing was performed only in my authorized lab.

20.4 What Not to Publish

- I do not publish private keys, .pem files, VPN keys, certificates containing private material, or password databases.
- I do not publish Keycloak client secrets, database credentials, API tokens, recovery codes, or MFA seeds.
- I do not publish raw firewall backups unless I have verified that secret material has been removed.
- I avoid publishing public IP information that I do not want permanently indexed.
- I sanitize screenshots so they do not reveal tokens, cookies, session identifiers, QR codes, secret environment variables, or personally sensitive data.

20.5 Project Overview for GitHub

I describe the project as a Zero Trust home lab that combines identity, device trust, network segmentation, policy-as-code, and centralized monitoring. The architecture and access matrix show how the main components interact. The two points I emphasize most are that a valid password from an untrusted device can still be denied, and that OPA and pfSense solve different problems: OPA evaluates contextual authorization while pfSense controls network reachability. Wazuh and Suricata give me the telemetry needed to verify and investigate those controls.

21. Resume and Interview Summary

21.1 Resume Project Title

NIST-Aligned Zero Trust Architecture Home Lab | pfSense, Keycloak, OPA, Wazuh, Suricata, Kali Linux

21.2 Resume Bullets

- Designed and implemented a NIST SP 800-207-aligned Zero Trust home lab using pfSense, Keycloak, Open Policy Agent, Wazuh, Suricata, and Kali Linux across segmented user, server, IoT, and management trust zones.
- Enforced least-privilege and default-deny access with inter-zone firewall controls, centralized OIDC authentication and MFA, device-aware authorization, and policy-as-code for protected application and management resources.
- Validated controls through authorized reconnaissance, stolen-credential, untrusted-device, direct-database-access, IoT-isolation, and lateral-movement test scenarios while centralizing firewall, identity, host, application, and IDS telemetry for investigation.
- Mapped the architecture to NIST SP 800-207 concepts and CISA Zero Trust Maturity Model pillars, documenting trust boundaries, access matrices, policy decisions, residual risks, and evidence requirements.

21.3 Short Resume Version

Built a NIST SP 800-207-aligned Zero Trust lab with pfSense microsegmentation, Keycloak MFA/OIDC, OPA policy-as-code, device-aware access, and Wazuh/Suricata monitoring; validated least-privilege controls using authorized Kali test scenarios.

21.4 Interview Talking Points

Question	My explanation
Why is this Zero Trust and not just VLANs?	I used segmentation as only one enforcement layer. Identity, MFA, device trust, resource context, OPA decisions, PEP enforcement, and telemetry also determine access.
What is the PDP and PEP?	OPA is my main policy-decision function. The reverse proxy/application and pfSense enforce decisions at the application and network layers.
What happens if a password is stolen?	MFA prevents password-only authentication from being sufficient, and sensitive authorization can still require a trusted device and the correct resource context.
What happens if a user laptop is compromised?	Inter-zone rules and per-resource authorization prevent a compromised endpoint from directly reaching the database or management plane.
Why Wazuh and Suricata together?	I use Wazuh to centralize host, identity, firewall, and application events, while Suricata adds network detection and scan/flow visibility.
What is the biggest limitation?	My biggest limitation is simplified device trust. A certificate proves device identity, but it does not provide the same continuous health posture as enterprise MDM/EDR.

21.5 Keywords for ATS / Portfolio

Zero Trust Architecture (ZTA), NIST SP 800-207, CISA ZTMM, network segmentation, microsegmentation, pfSense, IAM, OpenID Connect (OIDC), MFA, device trust, policy-as-code, Open Policy Agent (OPA), least privilege, default deny, policy decision point (PDP), policy enforcement point (PEP), Wazuh, SIEM, Suricata, IDS/IPS, centralized logging, threat modeling, lateral movement, security validation, Kali Linux.

Appendix A. Representative Configuration and Policy Excerpts

A.1 Network Plan

VLAN 10	USERS	10.0.10.0/24	
VLAN 20	SERVERS	10.0.20.0/24	
VLAN 30	IOT	10.0.30.0/24	
VLAN 40	MANAGEMENT	10.0.40.0/24	
VLAN 50	SECURITY	10.0.50.0/24	# optional

A.2 pfSense Rule Intent

USERS:

```
allow TCP 443 -> protected reverse proxy
allow required DNS/NTP
deny USERS -> DATABASE
deny USERS -> MANAGEMENT
deny remaining protected inter-zone traffic
```

SERVERS:

```
allow APP_HOST -> DATABASE:3306
allow required DNS/NTP/updates
deny SERVERS -> MANAGEMENT unless explicitly required
```

IOT:

```
allow only required Internet/DNS/NTP services
deny IOT -> USERS
deny IOT -> SERVERS
deny IOT -> MANAGEMENT
```

MANAGEMENT:

```
allow approved admin endpoint -> pfSense/Keycloak/Wazuh/SSH
deny ordinary client sources
```

A.3 OPA Policy Test Cases

```
# Test cases to encode in policy tests
admin + MFA + trusted device + management zone + ssh-management => allow
admin + MFA + untrusted device + management zone + ssh-management => deny
user + MFA + trusted device + user zone + web-app => allow
user + no MFA + trusted device + user zone + web-app => deny
user + MFA + trusted device + user zone + database => deny
```

A.4 .gitignore Security Entries

```
# Secrets and private key material
.env
*.pem
*.key
*.p12
*.pfx
*secret*
```



```

*credentials*

# Local exports/backups that may contain secrets
*.xml.bak
config.xml
backups/

# Keep sanitized examples explicitly
!.env.example
!docs/examples/**

```

A.5 Validation Record

Test ID: VT-06

Objective: Verify a user endpoint cannot directly connect to the database.

Source: User VLAN test host

Destination: Database service, TCP/3306

Precondition: User-facing web access remains functional.

Action: Attempt a TCP connection to the DB port.

Expected: Direct connection denied; application-mediated access remains functional.

Evidence: Client output + pfSense firewall log + timestamp.

Result field: documented alongside the corresponding test evidence in the repository.

Notes field: used to record rule changes, troubleshooting, and retest details.

Glossary

Term	Definition in this report
Zero Trust Architecture (ZTA)	Architecture that does not grant implicit trust solely because of network location and evaluates access to protected resources using explicit policy.
Identity Provider (IdP)	System that authenticates users and issues identity assertions/tokens; Keycloak in this lab.
MFA	Multi-factor authentication; requires more than one authentication factor.
OIDC	OpenID Connect, an identity layer built on OAuth 2.0 used here for application sign-in.
PDP	Policy Decision Point; evaluates policy and returns an authorization decision.
PEP	Policy Enforcement Point; enforces the allow/deny result for a protected resource.
OPA	Open Policy Agent; policy-as-code engine used as the lab PDP.
Microsegmentation	Fine-grained restriction of communication between systems/workloads rather than broad flat-network access.
Least privilege	Grant only the minimum access required for a user, device, or workload to perform its function.
Default deny	Access is denied unless an explicit policy permits it.
Device trust	A contextual signal that the endpoint is approved/known; simplified in this lab to an identity/certificate signal.
SIEM	Security Information and Event Management; Wazuh is used for centralized security event analysis.
IDS/IPS	Intrusion Detection/Prevention System; Suricata provides network security visibility and optional inline enforcement capability.
Lateral movement	Movement from one compromised host/resource to additional internal resources.
Trust boundary	Point where a request crosses between components or zones with different security assumptions and must be controlled.

References

- [1] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, National Institute of Standards and Technology, Aug. 2020. <https://doi.org/10.6028/NIST.SP.800-207>
- [2] National Institute of Standards and Technology, "Implementing a Zero Trust Architecture: High-Level Document," NIST SP 1800-35, final, June 2025. <https://doi.org/10.6028/NIST.SP.1800-35>
- [3] Cybersecurity and Infrastructure Security Agency, "Zero Trust Maturity Model Version 2.0," Apr. 2023. https://www.cisa.gov/sites/default/files/2023-04/zero_trust_maturity_model_v2_508.pdf
- [4] Netgate, "pfSense Documentation - Firewall," current documentation. <https://docs.netgate.com/pfsense/en/latest/firewall/>
- [5] Netgate, "VLANs and Security," pfSense Documentation. <https://docs.netgate.com/pfsense/en/latest/vlan/security.html>
- [6] Keycloak, "Server Administration Guide," current documentation. https://www.keycloak.org/docs/latest/server_admin/
- [7] Open Policy Agent, "OPA Documentation," current documentation. <https://www.openpolicyagent.org/docs>
- [8] Open Policy Agent, "How to Deploy OPA," current documentation. <https://www.openpolicyagent.org/docs/deploy>
- [9] Wazuh, "Log Data Analysis / Log Data Collection," current documentation. <https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/log-data-analysis.html>
- [10] Open Information Security Foundation, "Suricata Documentation - IPS Mode," current documentation. <https://docs.suricata.io/en/latest/ips/>